

万物皆网络，万字长文详解社区发现算法Leiden



小伍哥聊风控

风控算法、风控策略，反欺诈，异常检测，复杂网络挖掘、风控转行

收录于 · 风控策略算法 >

50 人赞同了该文章 >

大家好，我是小伍哥，今天继续聊聊社群检测算法。做风控的都知道，在互联网风控、金融风控等场景里，团伙识别是相当重要的一项工作，可以说是必不可少，并且现在的关系网络越来越复杂，黑灰产常常以团伙的方式尝试获取利益，比如诈骗、薅羊毛、刷单等，通常都会给公司带来不小的经济损失。前期写了louvain的



小伍哥聊风控：万物皆网络，万字长文详解
社区发现算法Louvain

409 赞同 · 62 评论 文章

团伙识别有各种各样的方法，其中最主要的方法就是【社区发现：community detection】类算法，常规的方法有Louvain，LPA，极大连通子图、Infomap等。

我们今天聊Leiden这个社群检测算法，[Leiden算法+](#)是2020年提出，目前[社区检测+](#)方法里面的SOTA之一，还是值得研究研究的。

论文标题：《From Louvain to Leiden: guaranteeing well-connected communities》，简单翻译下《从[Louvain算法+](#)到Leiden算法：保证连接良好的社区》。看标题就很清楚，Leiden算法是Louvain的改进算法。

当然，除了风控，在其他场景，社区发现算法也有很多的用处：

- 社交媒体网络分析，如识别兴趣小组或朋友圈
- 网络生物学研究，如在蛋白质相互作用网络中寻找功能模块、细胞聚类等
- 信息检索，用于在文档网络中找到主题聚类
- 甚至在网络路由优化、城市规划等领域也有潜在应用

Leiden这个算法，我目前还在实践探索中，先简单写写，先用起来再说，反正看实验数据，是比Louvain要优秀。

一、算法摘要

社区检测通常用于理解大型复杂网络的结构，用于揭示社区结构的最流行的算法之一是所谓的Louvain算法，我们证明了这个算法有一个主要的缺陷，直到现在才被忽视：Louvain算法可能产生任意连接不良的社区。

在最坏的情况下，社区甚至可能会断开连接，尤其是在迭代运行算法时。在我们的实验分析中，我们观察到多达25%的社区连接严重，最多16%的社区断开连接，这可能在随后的分析中出现严重问题，为了解决这个问题，我们介绍了莱顿算法。我们证明Leiden算法可以产生保证连接的社区。

此外，我们证明，当迭代应用莱顿算法时，它会收敛到一个分区，在该分区中，所有社区的所有子集都在本地进行了最佳分配。此外，通过依赖快速局部移动方法，Leiden算法比Louvain算法运行得更快。我们演示了Leiden算法在几个基准和现实网络中的性能。我们发现Leiden算法比Louvain算法更快，并且除了提供明确的保证外，还揭示了更好的分区。根据我们的结果，我们得出结论，Leiden算法优于Louvain算法。

Leiden算法有一个比较常用的得开源库，由Vincent Traag开发，在github叫做[LeidenAlg+](#)，它提供了高效的Python实现，用于执行社区检测任务，特别是在复杂网络中。

该项目使用了Cython进行性能优化，使得在大型网络上的运行速度非常快，此外，它还支持平行化处理，可以利用多核CPU来加速计算。

Leiden算法是对经典的Louvain算法的改进版，Leiden算法是为了改进Louvain算法的缺陷，Louvain算法可能会发现任意连接不良的社区，Louvain方法通过不断优化[模块度+](#)（Modularity）来找到网络的最佳划分，Leiden算法引入了一个更精细的终止条件，可以避免在

某些情况下出现过度分割的问题，从而产生更高质量的社区结构。既然是改进，那应该更优秀，后续会多实践应用下。



Leidenalg 软件包建立在 [igraph](#) 基础之上，有助于网络的社区发现，leiden 是大型网络中社区发现方法的通用算法。大家需要先安装好 [igraph](#)。

二、官方资料汇总

Leidenalg 的官方社区提供了丰富的资源，包括文档、教程和问题解答。如果你在使用过程中遇到问题，可以访问[Leidenalg 的 GitHub 页面]，github.com/vtraag/leidenalg，那里有一个问题跟踪器，你可以在那里提问或搜索已有的问题。

说明文档：[readthedocs.org/project...](https://readthedocs.org/project/leidenalg)

Github 链接：github.com/vtraag/leidenalg

论文链接：arxiv.org/pdf/1810.08473

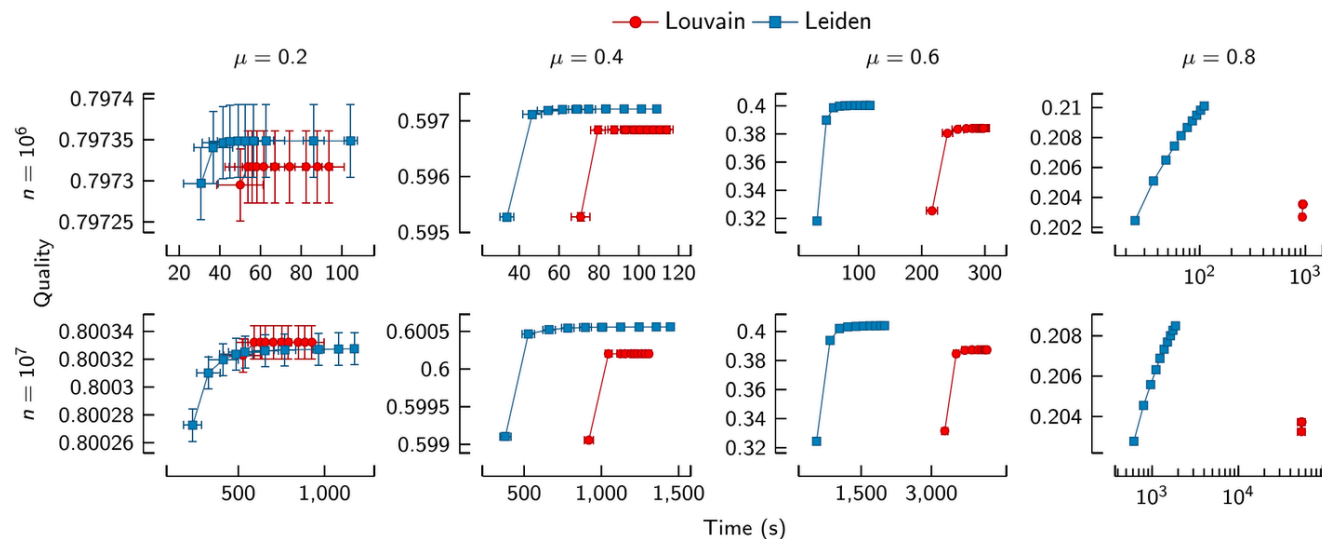
官方手册：[leidenalg.readthedocs.io...](https://leidenalg.readthedocs.io/)

官方文档：[leidenalg.readthedocs.io...](https://leidenalg.readthedocs.io/)

三、对 Louvain 的优化

1、运行速度更快

节点移动-->节点的本地移动：Louvain 算法对于本社区内的 每一个顶点都尝试和其它所有社区 进行模块度计算，而 Leiden 算法只针对的不稳定点 和它 直接相连的社区进行模块度计算。



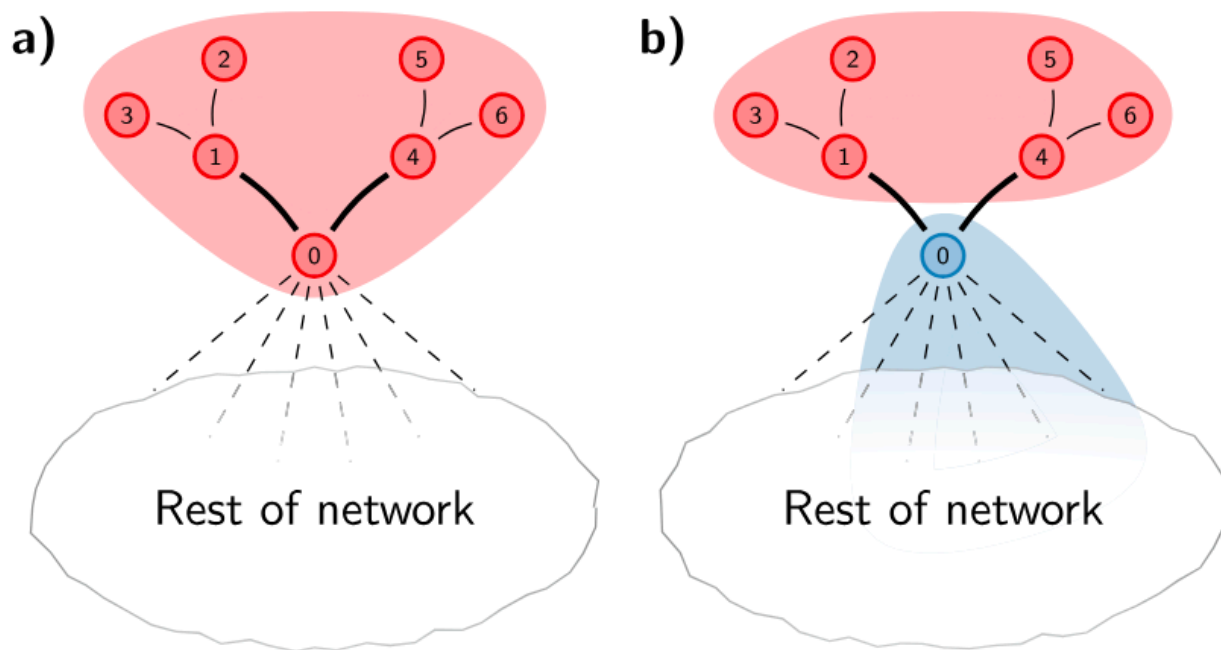
2、改善不良连接

Louvain 算法有一个主要的缺陷：可能产生任意非连通的社区，即可能会产生连接性差的社区，这些社区的节点之间可能不连通。

为了解决这个问题，V.A.Traag 等人提出了 Leiden 算法，该算法产生的社区保证是连通的[1]，并且 Leiden 算法是近几年社区发现领域中表现非常不错的算法之一。

具体来说，Louvain 算法在将节点分配到社区时，优先考虑节点与其所在社区内节点的连接强度，而不考虑节点与其他社区的情况。这容易使得一些节点被错误地分配到与其连接不紧密的社区，从而导致社区内部的连通性较差，某些社区甚至可能不连通。Rest of network: 网络的其余部分

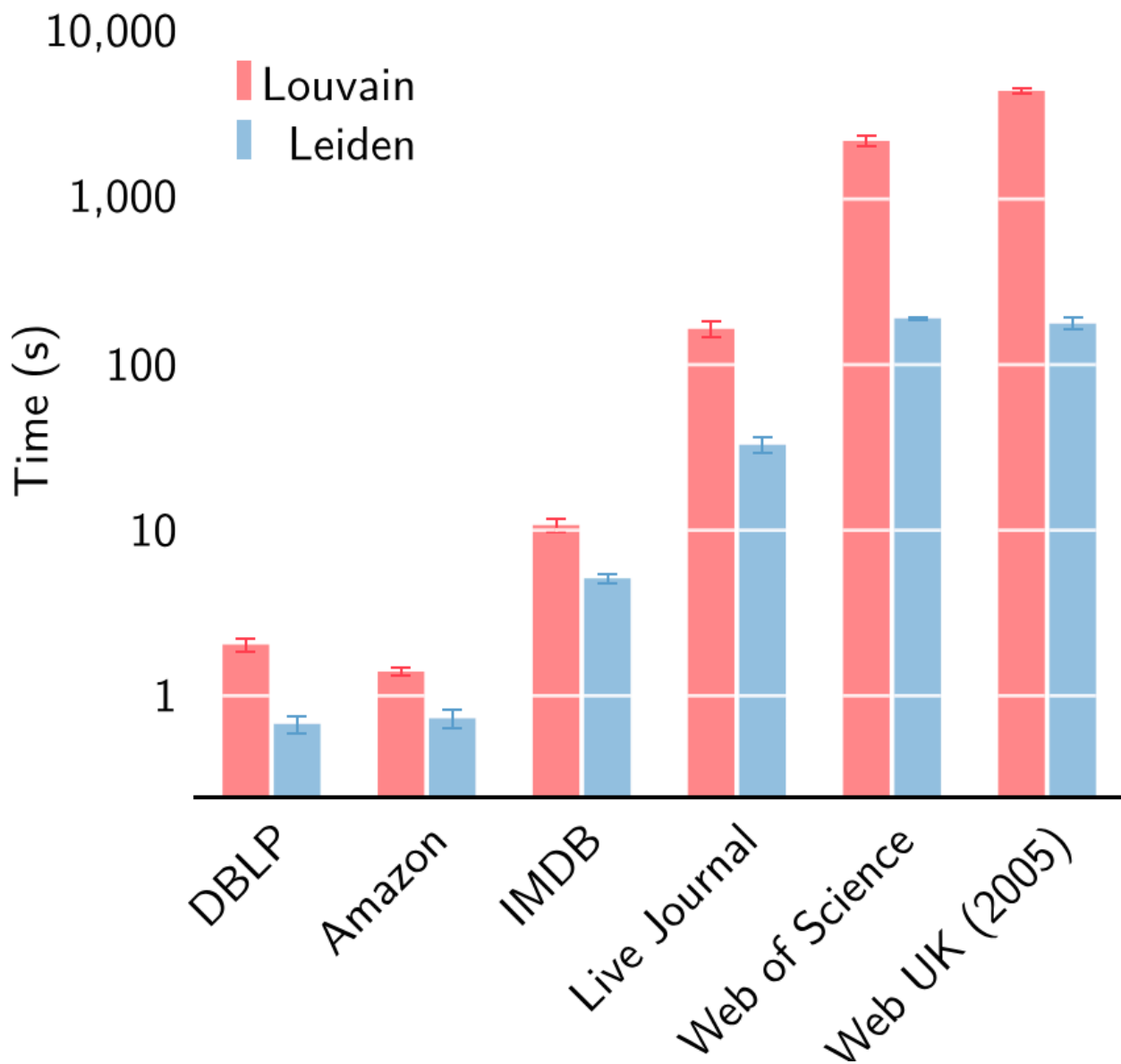
例如下图所示的情况：

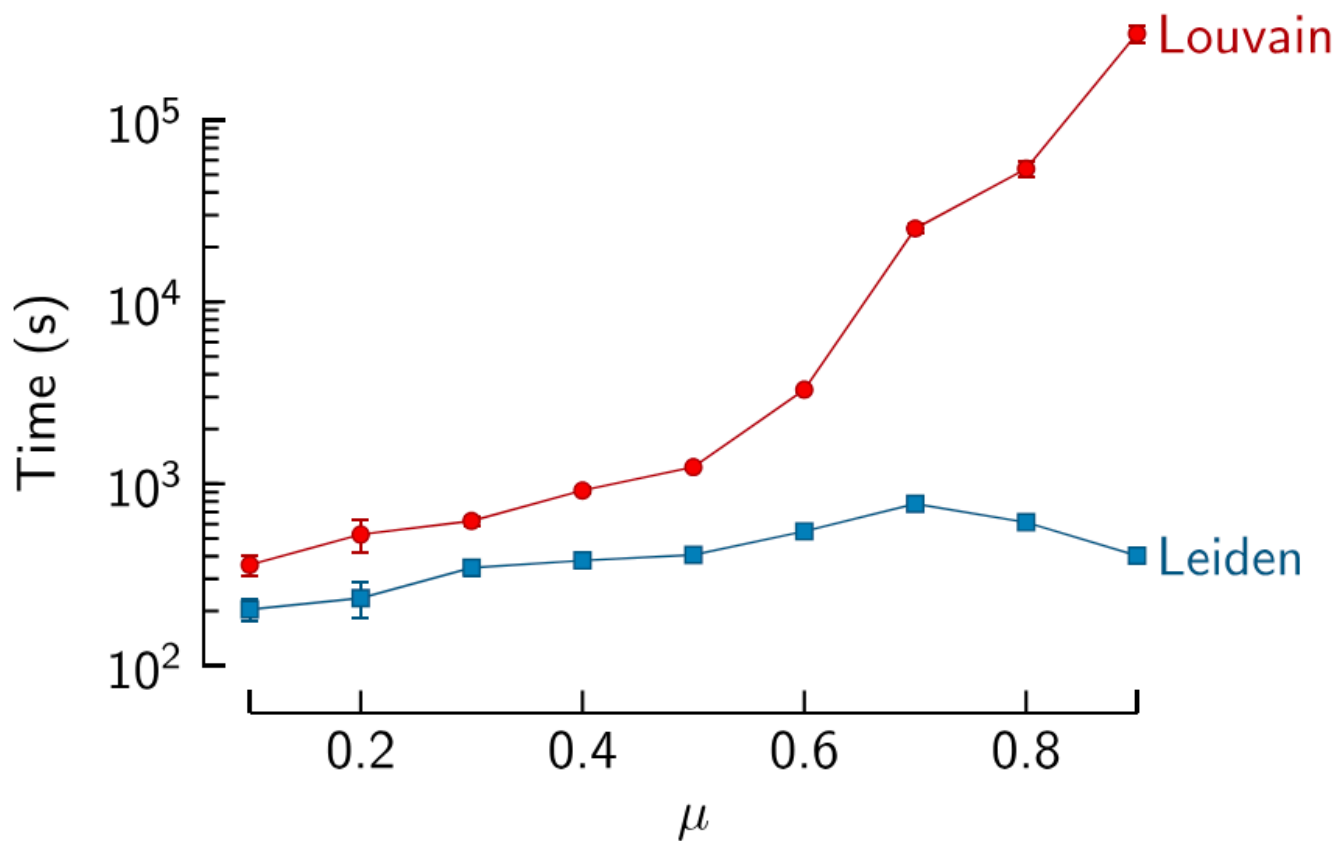


图中的粗边表示较强的连接，而其他边表示较弱的连接。如图 (b) 所示，当节点 0 移动到不同的社区时，原本图 (a) 中的社区(红色部分)内部断开，导致该社区不连通。显然，此时应当将节点 1-6 分为 1-3 和 4-6 两个社区。然而 Louvain 算法不考虑这种可能性，这是因为它只考虑单个节点的移动。

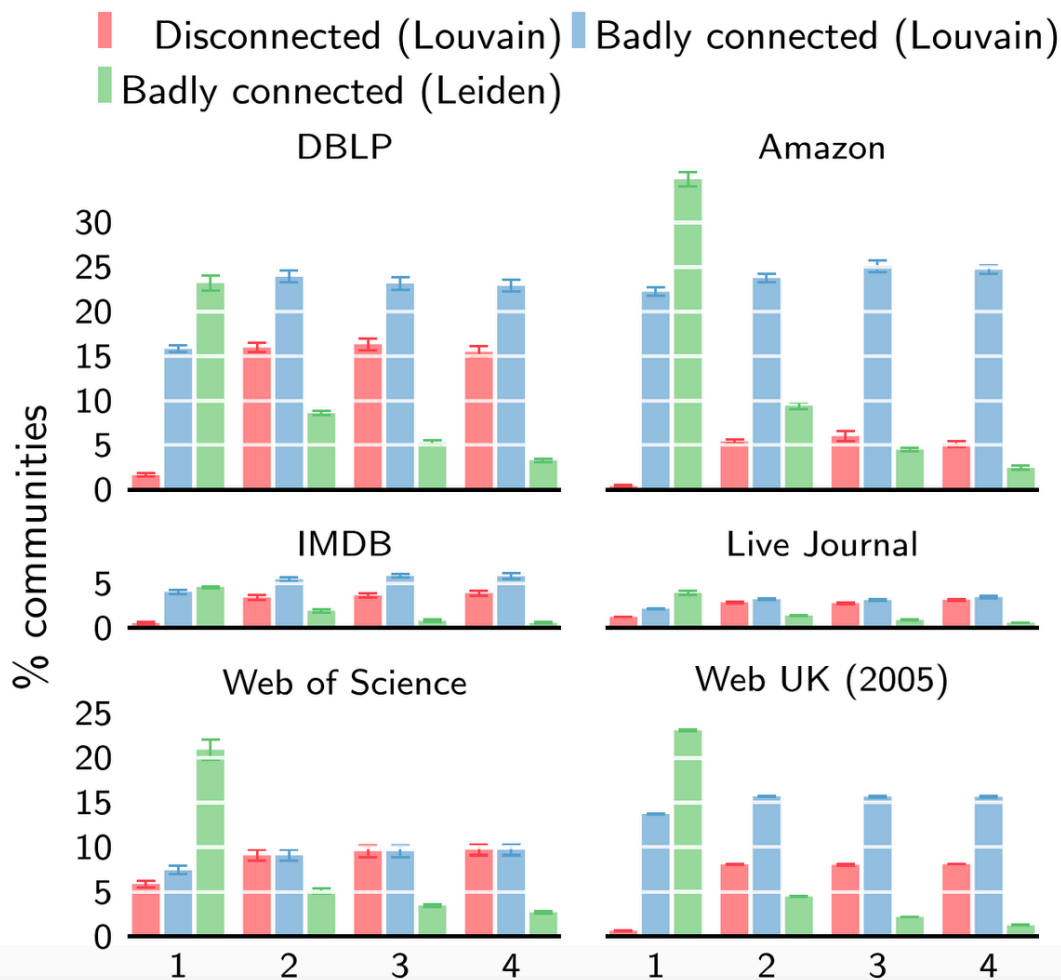
对于 Louvain 会产生非连通的社区的问题，根据图1我们做如下阐述。图1中的加粗边表示更强的连接，而其它边表示较弱的连接。在某个时刻，Louvain 算法可能会出现如图1 a)所示的社区结构。节点 0-6 位于同一社区中。节点 1-6 仅在此社区内具有连接，而节点 0 也具有许多外部连接。该算法继续移动网络其余部分中的节点。在某个时刻，节点 0 被认为是在移动。当节点 0 的足够数量的邻居已经在网络的其余部分中形成社区时，将节点 0 移动到该社区可能是最佳的，从而产生图1(b)所示的情况。

在图1(b)的情况下，节点 2、3、5 和 6 只有内部连接。因此，这些节点被最优地分配给它们的当前社区。在节点 0 被移动到不同的社区之后，此时节点 1 和 4 不仅具有内部连接，还具有外部连接。然而，根据不同连接的相对强度，这些节点仍然有可能最优地分配给它们的当前社区。在这种情况下，节点 1-6 都是本地最优分配的，尽管它们的社区已经断开连接。但通过直观探索图发现，1-6 分为两个图更好，节点 1-3 应该形成一个社区，节点 4-6 应该形成另一个社区。然而，Louvain 算法没有考虑这种可能性，因为它只考虑单个节点的移动，并且当不能移动更多的节点时，Louvain 开始进行社区压缩。当一个断开连接的社区成为聚合网络中的节点时，就没有更多的可能性来拆分社区。因此，社区仍然是断开的，除非它与另一个恰好充当桥梁的社区合并，显然，这是一个最坏的例子，表明断开连接的社区可能会被 Louvain 算法识别。总的来说，Louvain 可能会发现任意非连通的社区。





Leiden 算法能够分割集群 (即分区细化), 而不是像 Louvain 算法那样只合并它们。通过以特定方式拆分集群, Leiden 算法保证了集群之间的良好连接和 子集最优特性(即不可能通过将一个或多个节点从一个集群移动到另一个集群来提高集群的质量 -->通过模块度衡量)



四、算法原理详解

本文介绍的Leiden算法，它保证了社区之间良好连接，模块度（Modularity）的概念在Louvain算法中有所介绍，Leiden算法使用的是扩展的模块度公式，以处理不同密度的社区：

在Louvain算法里面给手动加个参数

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j),$$

论文里面的公式

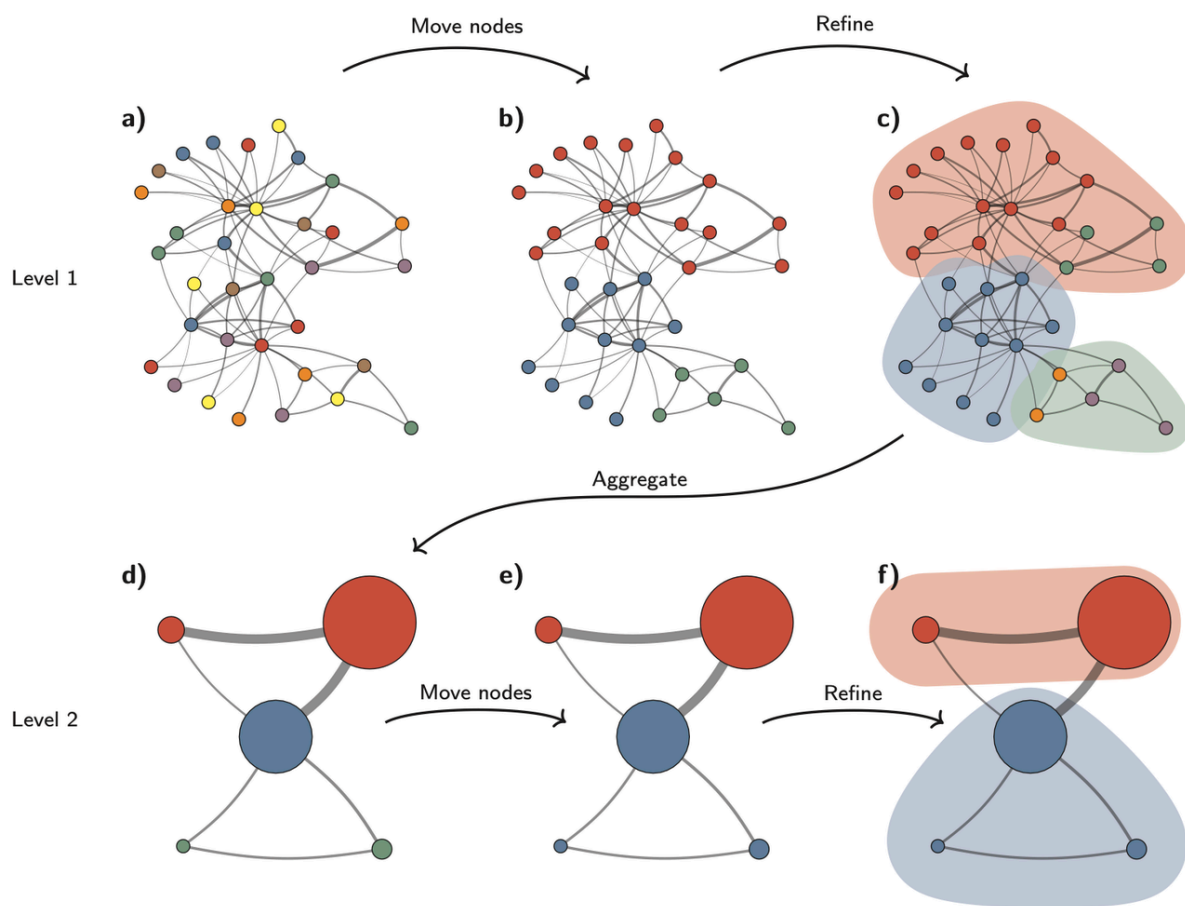
$$\mathcal{H} = \frac{1}{2m} \sum_c \left(e_c - \gamma \frac{K_c^2}{2m} \right)$$

$\gamma > 0$ 是调节社区内部和社区之间连接密度的分辨率参数

当 $\gamma > 1$ 时，会形成更多、更小且连接紧密的社区；

当 $\gamma < 1$ 时，会形成更少、更大但连接相对不那么紧密的社区。

Leiden算法由三个阶段组成：1) 局部移动节点；2) 社区优化；3) 社区压缩



Leiden算法开始时，图中的每个节点都在自己的社区中，然后算法进行多次迭代，Leiden 算法从单节点社区划分 a) 开始。

- a) 每个节点单独作为一个社区
- b) 首先进行节点的移动以得到社区划分
- c) 接着进行Refine，得到细化的社区划分
- d) 然后基于细化划分创建聚合网络，并使用非细化划分创建聚合网络的初始划分，例如，b) 中的红色社区被细化为 c) 中的两个子社区，在聚合后成为它们变成了 d) 中的两个独立节点，这两个节点都属于同一个社区
- e) 进一步，Leiden 移动聚合网络中的各个节点 e)在上图的情况下，细化不改变社区划分 f)
- 重复这些步骤，直到模块度没有进一步的提升

由上图可以看出，对比 louvain 算法，Leiden 算法主要是添加了 Refine 步骤对划分结果进行改善，防止社区内部出现不良连接。迭代过程核心分为下面三个过程。

1、局部移动节点

Louvain 算法在第一阶段会一直遍历图中的所有节点，直到没有任何节点移动能增加模块度。Leiden 算法则采用一种更高效的方法，它遍历完图中所有的节点一次后，只会访问那些邻居节点发生了变化的节点。为了实现这一点，Leiden 算法使用一个队列(先进先出)，初始化时按随机顺序将图中的所有节点添加到队列中。然后，重复下面的步骤直到队列为空：

- 从队列最前端移除一个节点。
- 将该节点分配至能获得最大模块度增益 (ΔQ) 的不同社区；如果加入任何社区都不能获得大于 0 的增益，则保留该节点在原社区。
- 如果该节点被移动到另一个社区，将所有不属于该节点新社区且不在队列中的邻居节点添加到队列尾端(因为要访问所有变化了的邻居节点)。

第一阶段以原图的一种社区划分 P 结束。

2、社区优化(Refine 步骤)

refine：精炼;改进;改善;提纯;使精练;去除杂质；

本阶段旨在对社区 P 进行优化以获取 P_{refined} ，以确保所有的社区有良好的连通性(b 阶段改善为 c 阶段, 节点所属的社区并没有改变)。初始化时， b 阶段中的每个节点都在自己的社区中，接着按如下方式 $\langle 1 \rangle \langle 2 \rangle$ 对 P 中的每个社区进行优化。优化阶段不遵循贪心方法，并且可能将节点与随机选择的社区合并，从而增加质量函数。

在这个阶段，每个节点并不一定会贪心地与产生最大模块度增益的社区合并。然而，与某个社区合并产生的增益越大，该社区被选择的可能性就越大。选择社区的随机程度由参数 $\theta > 0$ 决定，在社区合并过程中引入随机性，可以更广泛地探索分区空间。

在完成优化阶段后， P 中的社区通常会被拆分为优化后的多个社区。

3、社区压缩

例如在 b 中的红色社区被改善为 c 中的两种节点，但都属于一个社区，算法压缩后形成两个节点。然后算法继续移动凝聚后的网络节点，这个时候， f 不会再改善分区结果。完成第三阶段后，对聚合图进行迭代，迭代会一直进行，直至没有任何变化，达到最大的模块度。

所以我们可以看到在两种算法步骤中，主要的不同就是 Leiden 算法添加了 Refine 步骤对划分结果进行改善，防止社区内部出现不良连接。

4、算法优缺点

Leiden 算法是一种用于检测社区结构的社区发现算法，其优缺点如下：

1) 优点

- 高效性：Leiden 算法在处理大规模图时表现出良好的效率，可以快速找到图中的社区结构。
- 可扩展性：该算法可以处理具有不同分辨率参数的图，并且能够适应不同类型的网络结构。
- 鲁棒性：Leiden 算法对于噪声和局部连接的影响相对较小，能够在一定程度上保持稳健性。

2) 缺点

- 分辨率参数选择：Leiden 算法对于分辨率参数的选择比较敏感，不同的参数可能导致不同的社区结构，因此需要谨慎选择参数。
- 局部最优解：算法可能陷入局部最优解，特别是在处理具有多个重叠社区的复杂网络时，可能无法找到全局最优解。
- 对初始分区敏感：Leiden 算法对初始分区的选择比较敏感，不同的初始分区可能导致不同的结果，因此需要对初始分区进行一定的处理和调整。

总体来说，Leiden 算法是一种高效且鲁棒的社区发现算法，但在参数选择和初始分区处理上需要注意一些问题。

Leiden算法的核心在于其迭代过程和局部最优搜索策略。首先，它会将每个节点视为一个单独的社区，并计算当前的模块度。然后，它会在每次迭代中尝试移动节点到另一个社区，以最大化局部模块度增益。如果这种移动导致全局模块度提高，那么就接受这次移动。关键的区别在于，Leiden算法在每个阶段结束时还会检查整体的模块度，如果无法再提高，则停止迭代，而不是简单地基于局部最优。

五、算法的应用

1、算法安装

简单来说，可以使用 `pip install leidenalg` 直接安装。也可以使用源码进行安装，安装这个包需要 C 核心库 `igraph` 和 python 包 `python-igraph`，然后可以通过 `python setup.py test` 安装。需要安装下面的库才能实现。

```
pip install cairocffi
pip install leidenalg
pip install python-igraph
```

2、基础应用

Leidenalg 库的核心功能是通过 `find_partition` 函数来找到网络的社区。这个函数接受一个图作为输入，并返回一个社区的分配列表。

```
leidenalg.find_partition(graph,
                          partition_type,
                          initial_membership=None,
                          weights=None,
                          n_iterations=2,
                          max_comm_size=0,
                          seed=None,
                          **kwargs
)
```

参数解释：

- `graph` (`ig.Graph`)：需要进行社区发现的目标图数据结构，该参数类型应为 `ig.Graph`。
- `partition_type` (`type of :class:`)：用于优化的社区划分类型，例如 `ModularityVertexPartition`（模块度优化）或 `CPMVertexPartition`（CPM 算法），`MutableVertexPartition`、`SurpriseVertexPartition`、`RBERVertexPartition` 等
- `initial_membership` (`list of int`)：初始社区划分，以整数列表形式提供，列表长度应与图中节点数相同，每个元素代表对应节点所属的社区编号。如果为 `None`，则默认为每个节点各自形成一个独立社区。
- `weights` (`list of double, or edge attribute`)：边的权重。可以是一个浮点数列表，也可以是图对象的边属性名称。如果不指定，则默认所有边的权重为 1。
- `n_iterations` (`int`)：Leiden 算法的迭代次数。默认值为 2。如果设置为负数，则算法会一直运行，直到某次迭代不再改进社区结构为止。
- `max_comm_size` (`non-negative int`)：社区大小的最大限制，即一个社区最多允许包含的节点数量。默认为 0，表示对社区大小没有限制。
- `seed` (`int`)：随机数种子，用于确保算法结果的可重复性。默认情况下，如果不指定种子，则会使用随机种子。
- `kwargs`：传递给 `partition_type` 构造函数的其他参数，例如 `resolution_parameter`（分辨率参数）。

多路复用图(多元异构图)上的社区发现

```
leidenalg.find_partition_multiplex(graphs, partition_type,
                                   layer_weights=None, n_iterations=2,
                                   max_comm_size=0, seed=None, **kwargs)
```

时序网络上执行社区发现

```
leidenalg.find_partition_temporal(graphs, partition_type, interslice_weight=1,
                                  slice_attr='slice', vertex_id_attr='id', edge_type_attr='type', weight_attr='weight',
```

```
n_iterations=2, max_comm_size=0, seed=None, **kwargs)
```

部分案例如下。

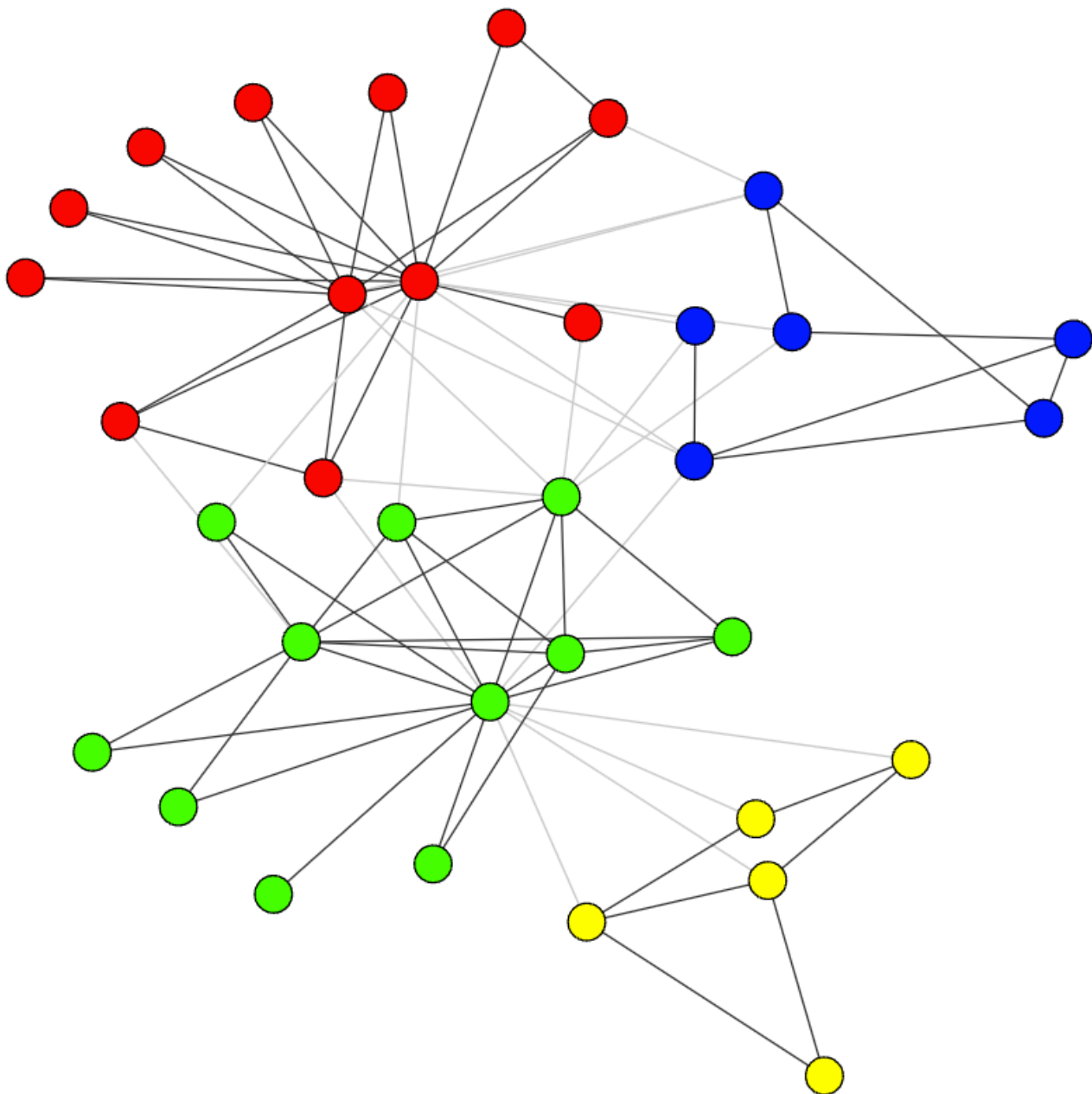
```
import numpy as np
import igraph as ig
import leidenalg as la

# 创建一个简单的网络
g = ig.Graph.Famous('Zachary')

# 计算社区分配
partition = la.find_partition(g, la.ModularityVertexPartition)

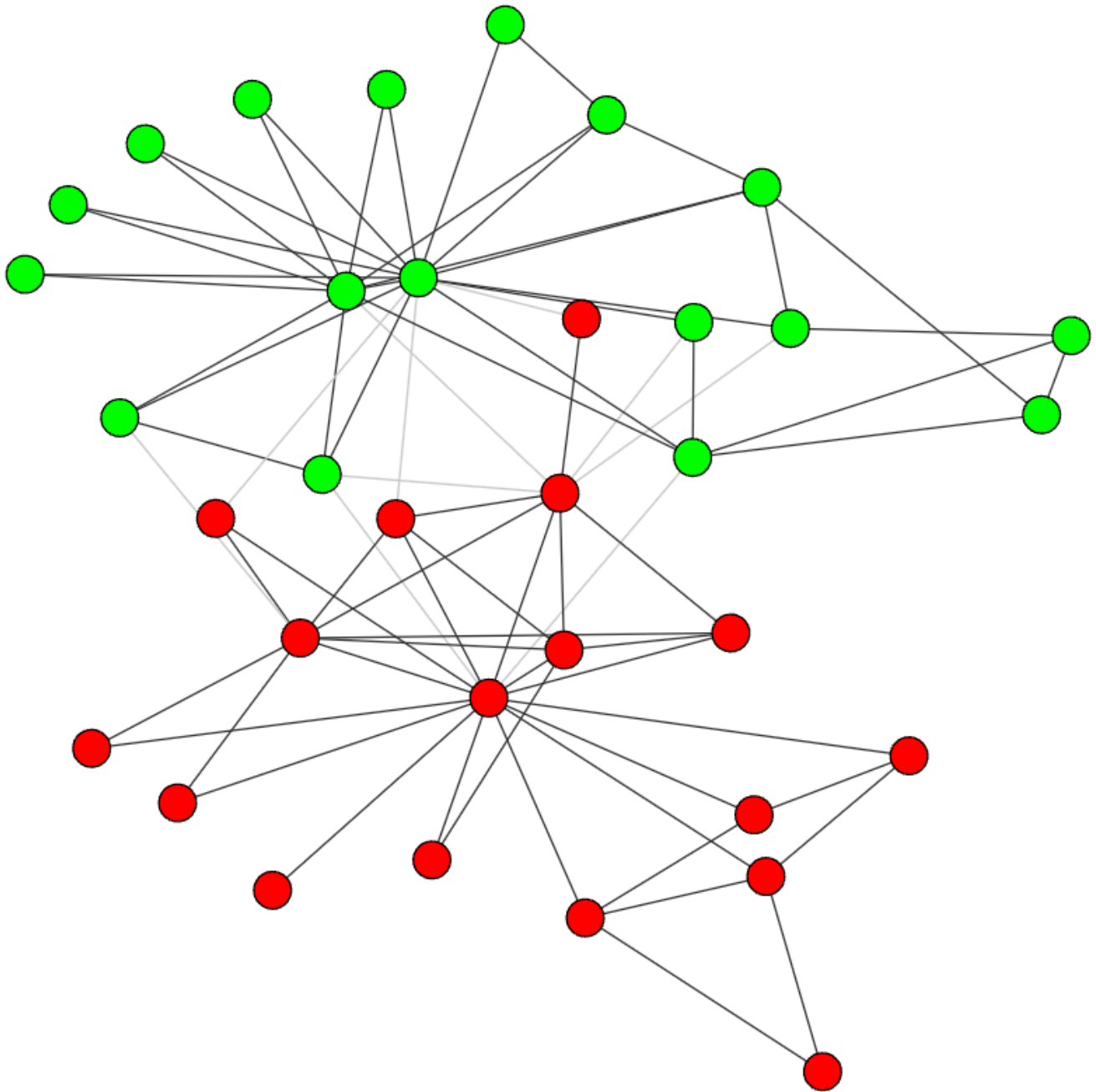
# 打印社区分配结果
print(partition)
Clustering with 34 elements and 4 clusters
[0] 8, 9, 14, 15, 18, 20, 22, 26, 29, 30, 32, 33
[1] 0, 1, 2, 3, 7, 11, 12, 13, 17, 19, 21
[2] 23, 24, 25, 27, 28, 31
[3] 4, 5, 6, 10, 16

# 简单的可视化
ig.plot(partition)
```



我们也可以换个算法看看 `la.CPMVertexPartition`。除了上面的两个方法，还有：`MutableVertexPartition`、`CPMVertexPartition`、`SurpriseVertexPartition`、`RBERVertexPartition`等

```
partition = la.find_partition(g,
                             la.CPMVertexPartition,
                             resolution_parameter = 0.05
                             )
ig.plot(partition)
```



3、分辨率调整

Leidenalg 还提供了其他一些高级功能，比如优化模块度的参数调整，通过设置 `resolution` 参数，可以控制社区的大小和精细度。

```
# 使用不同的分辨率参数
partition_high = find_partition(g, bin_method='infomap', resolution=0.3)
partition_low = find_partition(g, bin_method='infomap', resolution=1.0)

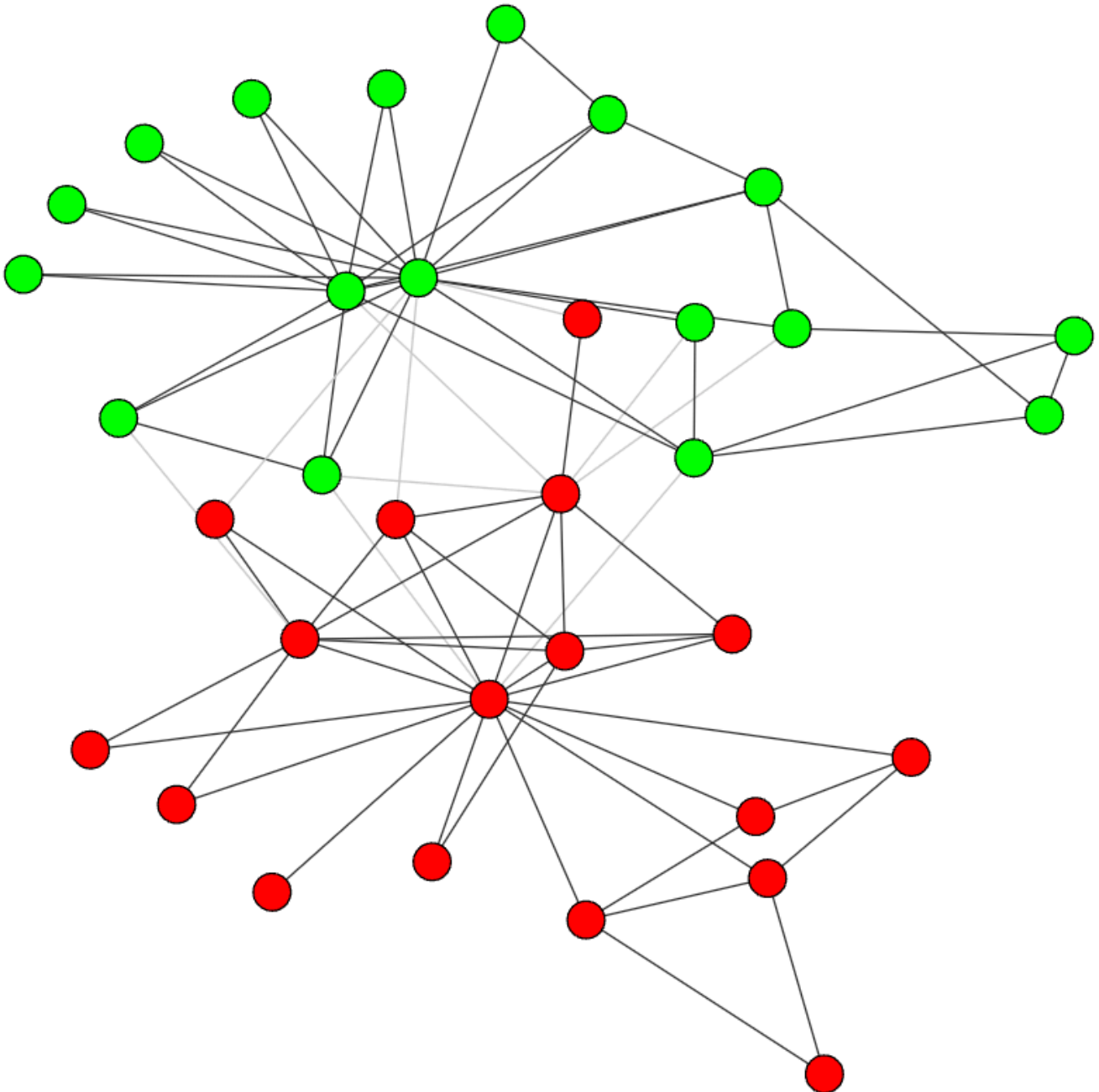
# 打印不同分辨率下的社区分配结果
print("High resolution partition:")
print(partition_high)
print("Low resolution partition:")
print(partition_low)
```

在上面的代码中，较低的分辨率值(0.3)会产生更细粒度的社区划分，而较高的分辨率值(1.0)会产生更大更少的社区。

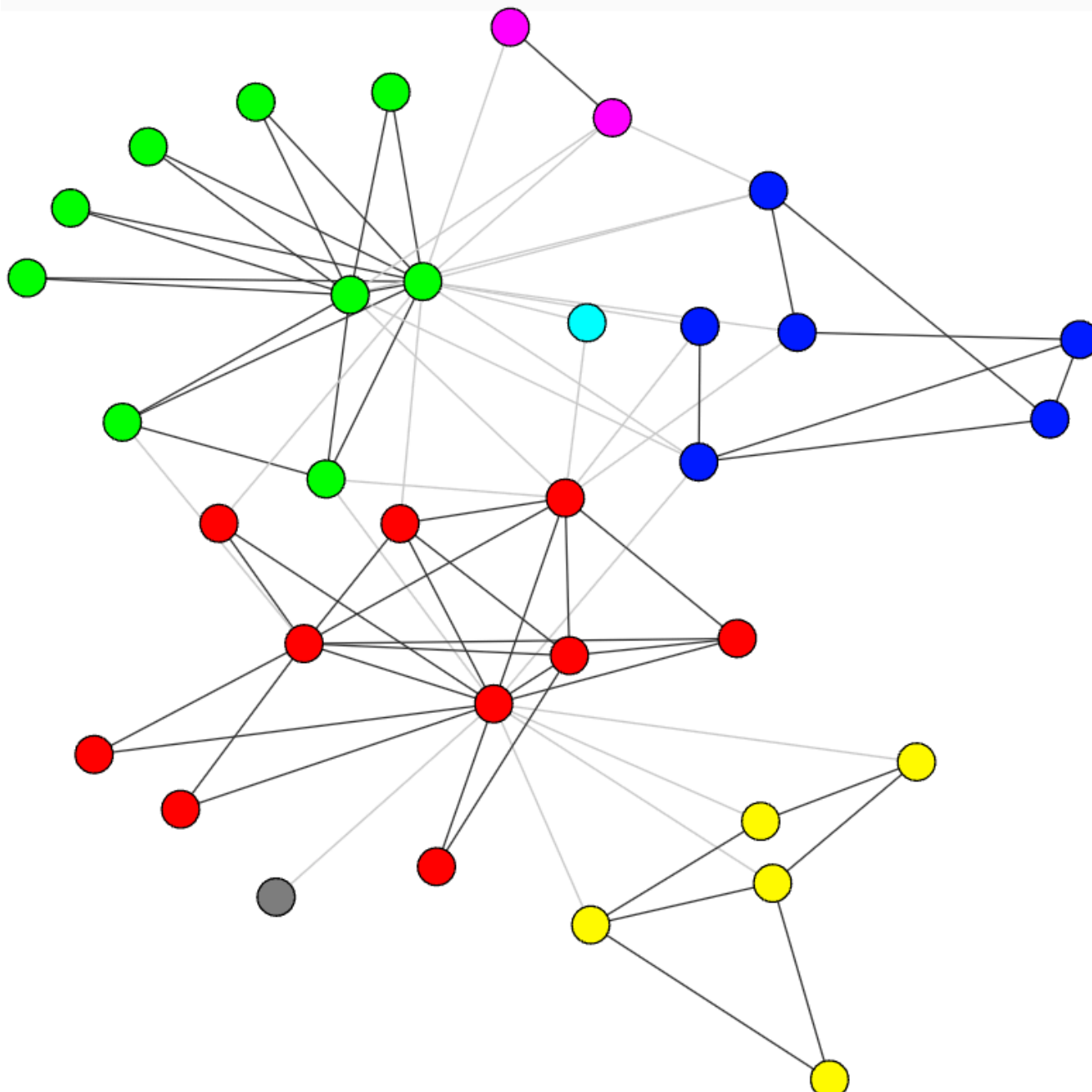
不同的分辨率可以得到不同的社区粒度

```
partition = la.find_partition(g,  
                             la.CPMVertexPartition,  
                             resolution_parameter = 0.05  
                             )  
  
ig.plot(partition)
```

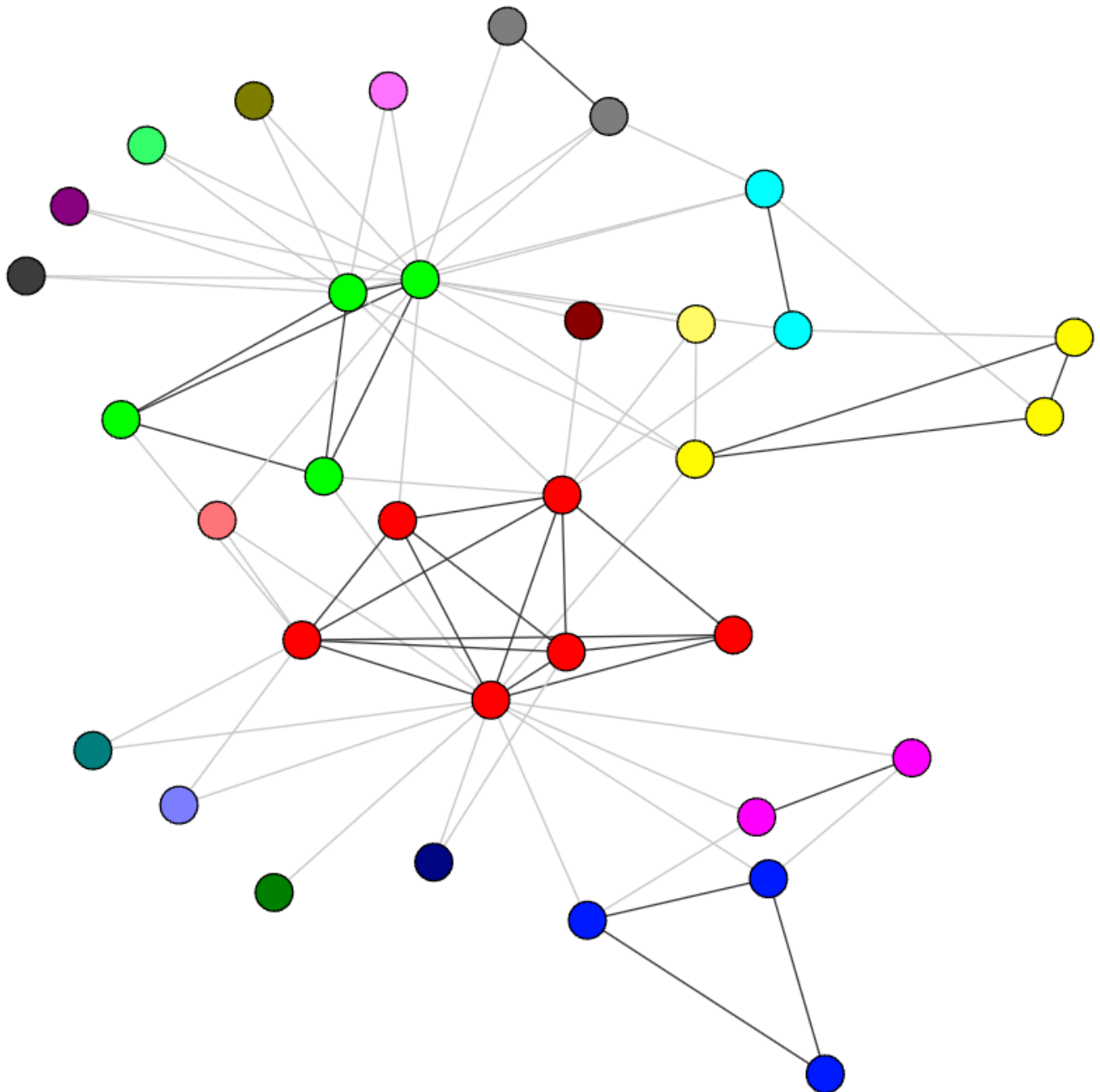
resolution_parameter = 0.05



resolution_parameter = 0.2



resolution_parameter = 0.6



4、限制社区规模

在某种情况下，可能需要限制最大子社区的规模，可以通过`max_comm_size`参数来指定最大子社区规模。此参数可以在优化期间进行设定产生约束，也可以直接传递到`find_partition()`。

```
# 限定最大子社区规模为10
partition = la.find_partition(G,
                             la.ModularityVertexPartition,
                             max_comm_size=10
                             )
```

5、提取子图

我们进一步研究下子图提取的细节，换个比较简单的数据集，提取子图：根据聚类结果，提取属于特定社区的节点和边，构成子图。

```
import leidenalg as la
import igraph as ig

# 创建一个示例图
g = ig.Graph.Famous("Zachary")
#print(g)

# 应用 Leiden 算法进行社区检测
partition = la.find_partition(g, la.ModularityVertexPartition)

# 打印每个节点所属的社区
print(partition.membership)

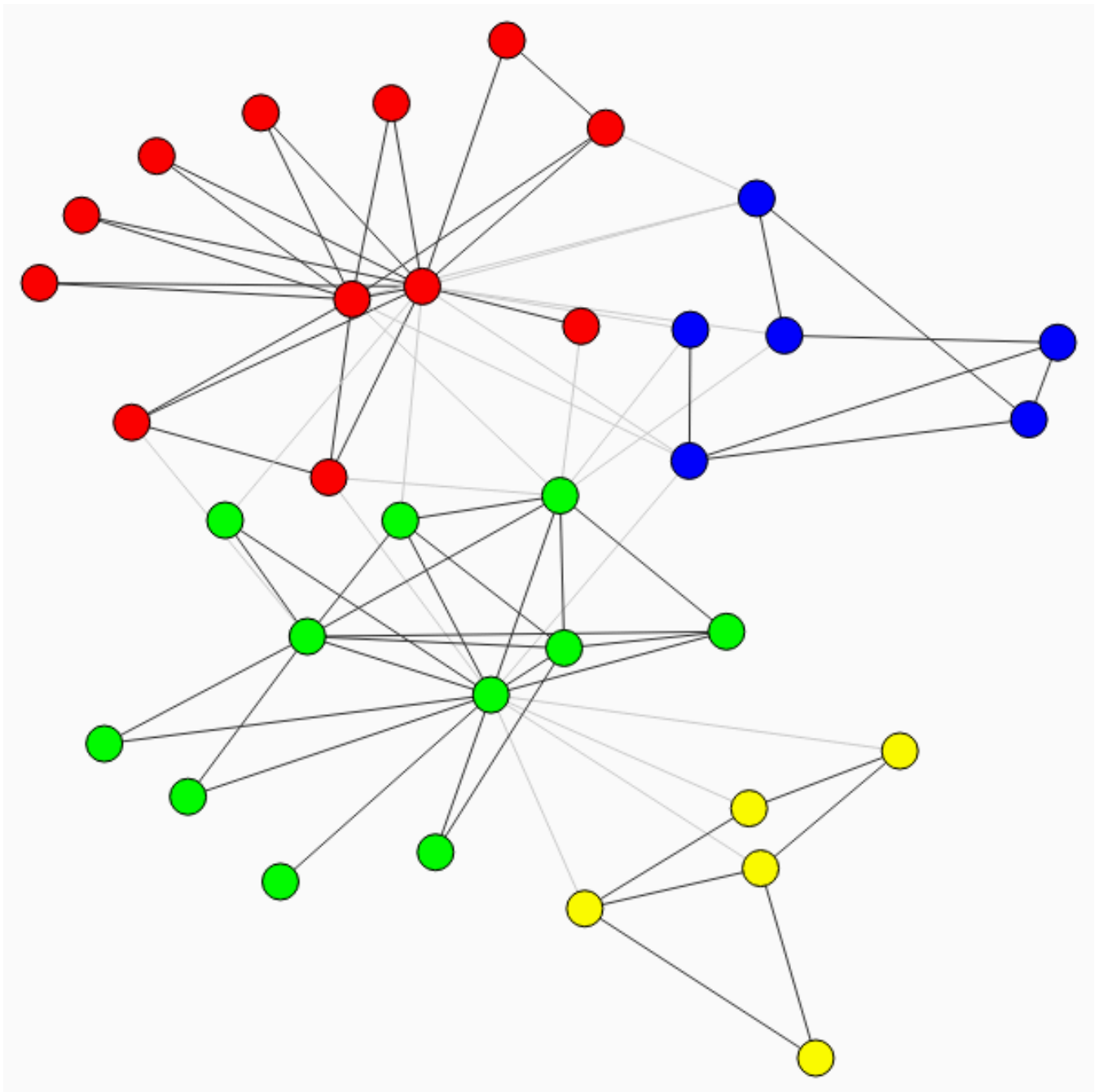
# 提取第一个社区作为子图
subgraph = g.subgraph(partition[0])

# 打印子图信息
print(subgraph.vcount()) # 节点数量
print(subgraph.ecount()) # 边数量

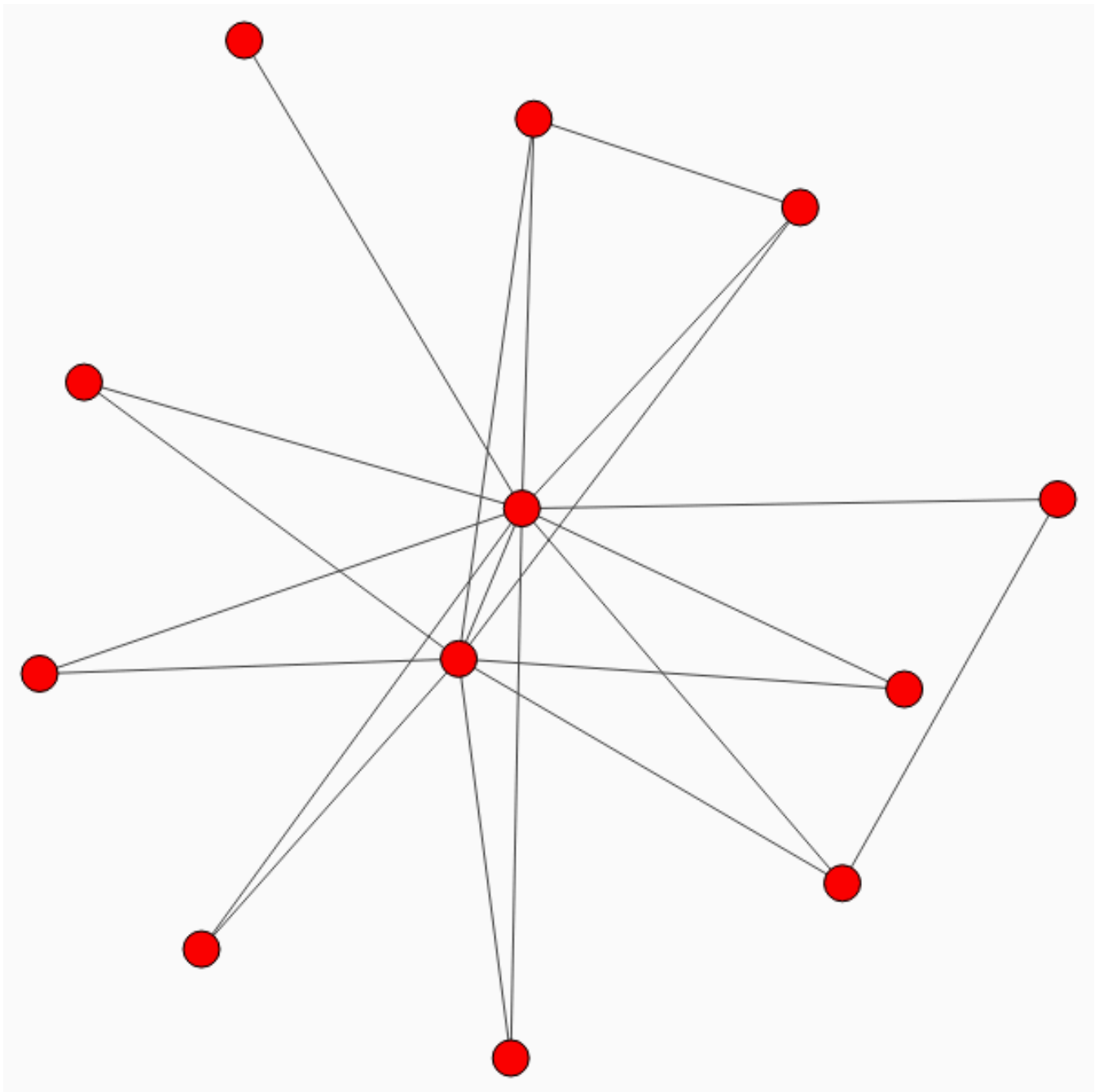
# 全图可视化
ig.plot(partition)

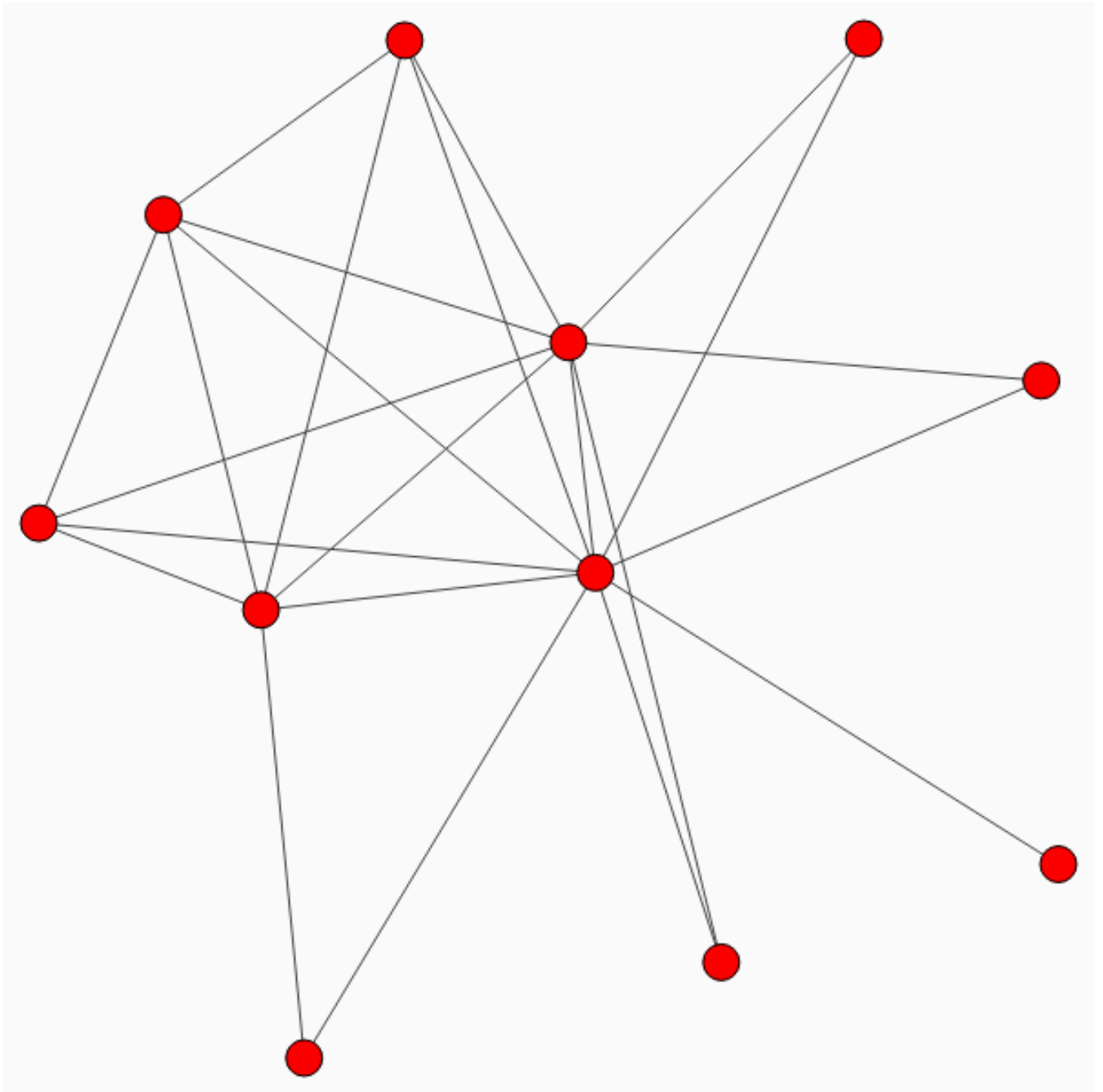
# 子图可视化
ig.plot(g.subgraph(partition[3]))
```

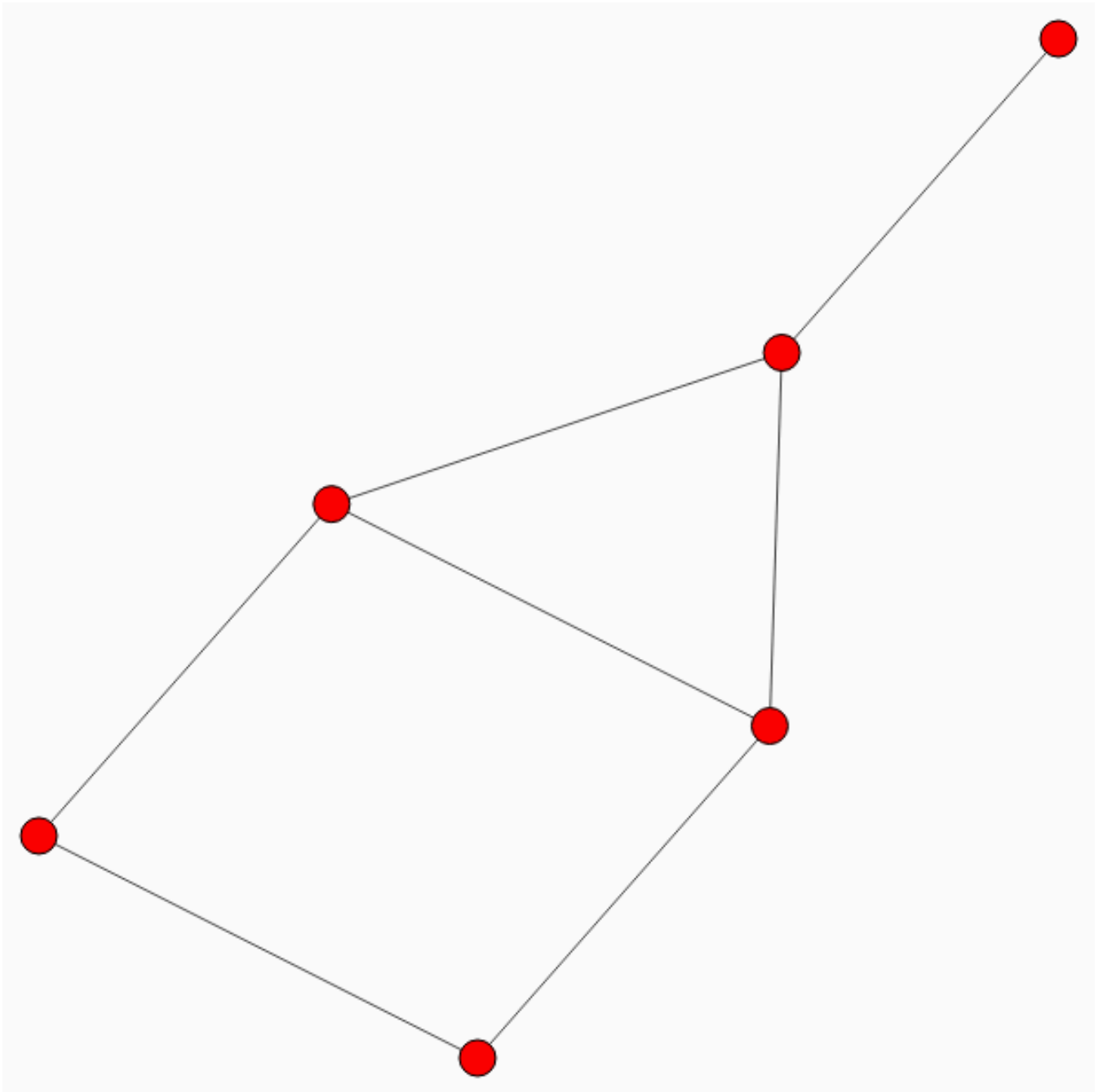
可视化：原始的社群

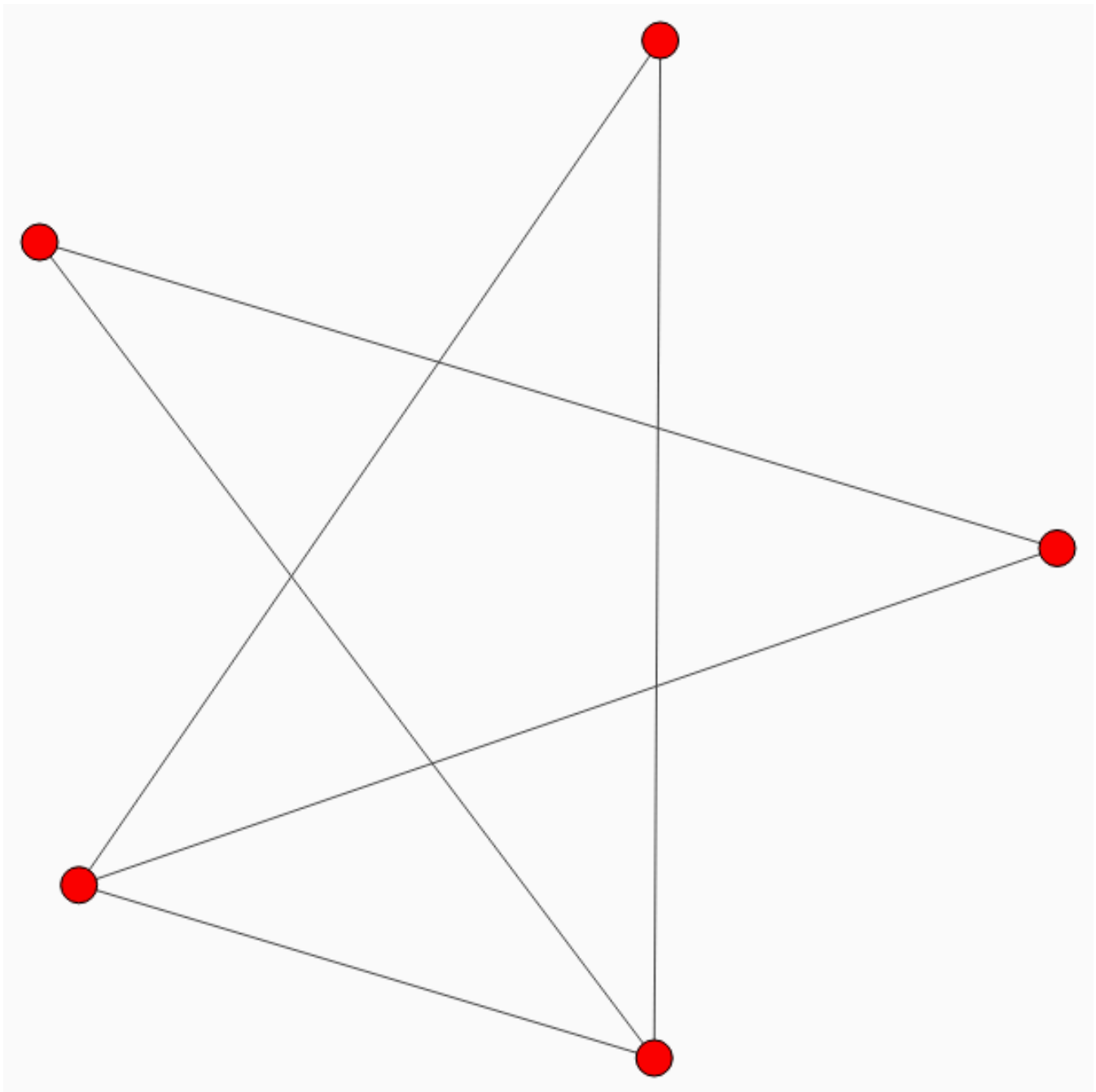


可视化：切分后的4个子图









模块度计算

```
#假设你已经有了 leiden 分区的结果, 并且 G 是一个 igraph.Graph 对象
modularity = partition.quality()
modularity
0.4197896120973044
```

6、增量更新

由于某些原因, 只更新分区(子社区)的一部分可能是有益的, 例如, 在一些数据上已经运行了leiden算法, 并对结果进行了一些分析, 随后又收集了一些新的数据, 特别是新的节点, 那么保持以前的社区分配不变, 而只更新新节点的社区分配, 这样做可能是有益的。这个点我还没时间细细研究, 后面在探索下写。

可以使用 `is_membership_fixed` 参数传入 `find_partition()` 来完成分区。

例如, 假设我们之前检测到图G的分区, 它被扩展到图G2。假设之前退出的节点是相同的, 我们可以通过以下方式创建一个新的分区:

```
new_membership = list(range(G2.vcount()))
new_membership[:G.vcount()] = partition.membership
```

然后，我们只能按照如下方式更新新节点的社区分配

```
new_partition = la.CPMVertexPartition(G2,
                                       new_membership,
                                       resolution_parameter=partition.resolution_parameter
                                       )
is_membership_fixed = [i < G.vcount() for i in range(G2.vcount())]

diff = optimiser.optimise_partition(partition,
                                     is_membership_fixed=is_membership_fixed
                                     )
```

在这个例子中，使用了CPMVertexPartition，也可以使用其他VertexPartition。

六、Leiden算法实战

电子邮件网络基于德国基尔大学 112 天收集的流量数据。每个节点都是一个电子邮件地址，如果 A 向 B 发送了至少一封电子邮件，则从节点 A 到节点 B 有一个定向链接。

1、数据读取

```
#读取前两列
import networkx as nx
import pandas as pd
import numpy as np
import igraph as ig
import leidenalg as la

data = pd.read_csv('email.edgelist.txt',delimiter='\t',header=None)
data = data.iloc[1:,:2]
data.columns = ['src','dst']
data
```

2、数据构图

这里的构图，需要按照igraph的构图思路做，具体如下

```
# 创建一个igraph的空图
g = ig.Graph()
da = data[['src','dst']].values

# 边构建
edges = []
for num in range(len(da)):
    edges.append((da[num,0],da[num,1]))

# 节点构建
nodes = set(data['src'].astype('str').tolist()+data['dst'].astype('str').tolist())

for node in nodes:
    g.add_vertex(node)

g.add_edges(edges)
```

3、社群检测

```
# 计算社区分配
partition = la.find_partition(g, la.ModularityVertexPartition)
```

格式整理

```
# 创建一个新的DataFrame来存储节点Label及其模块化社区编号
com = pd.DataFrame({"group_id": g.vs['name'],
                    "Community": partition.membership
                    })

com.head()
```

统计每个群组人数

```
# 统计每个群组人数
com.groupby('GroupID').count().sort_values(by='User_ID', ascending=False) .head(120)

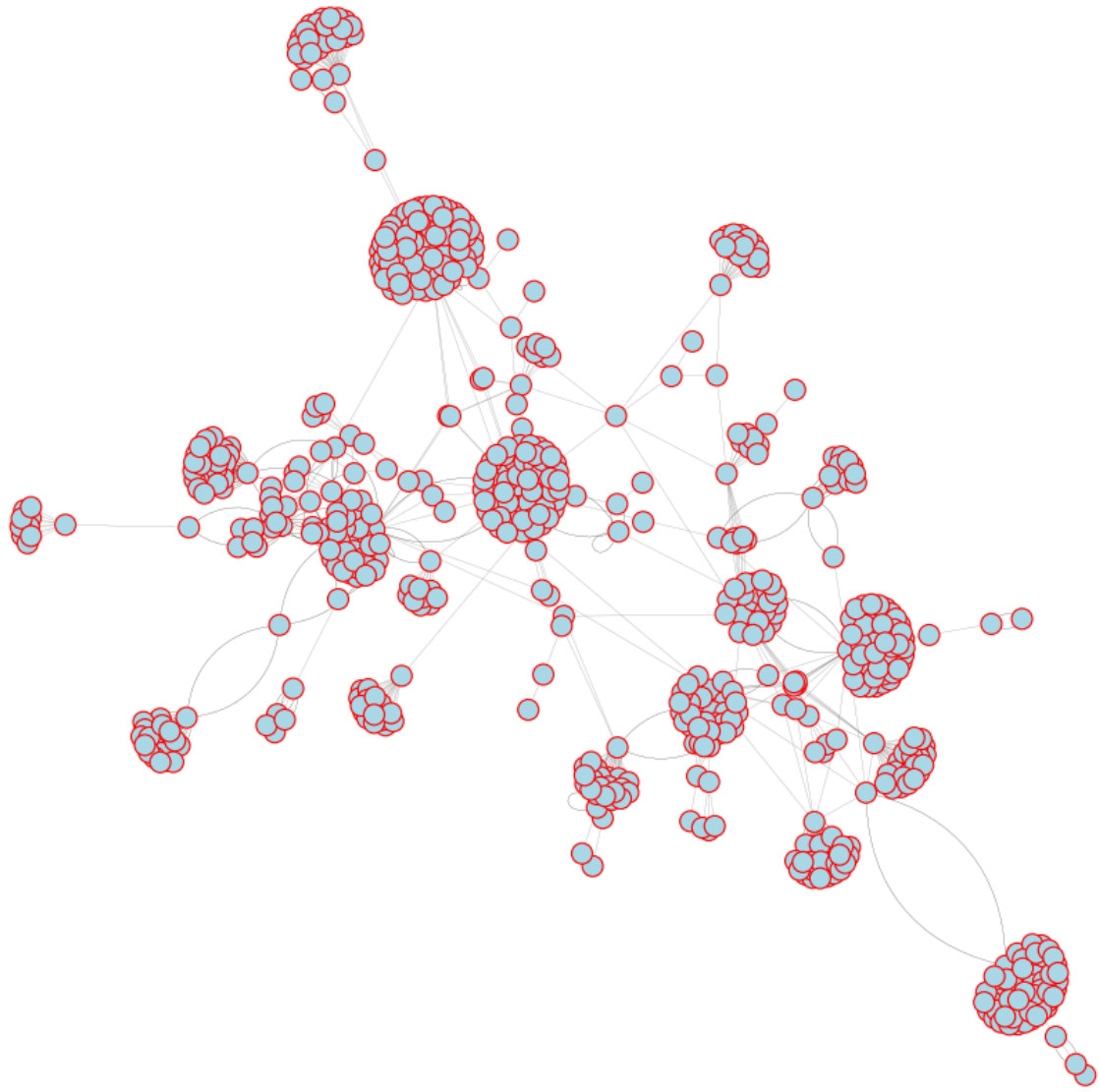
# 也可以提取子图看看
sub_g = partition.subgraph(112)
sub_g.vcount()
```

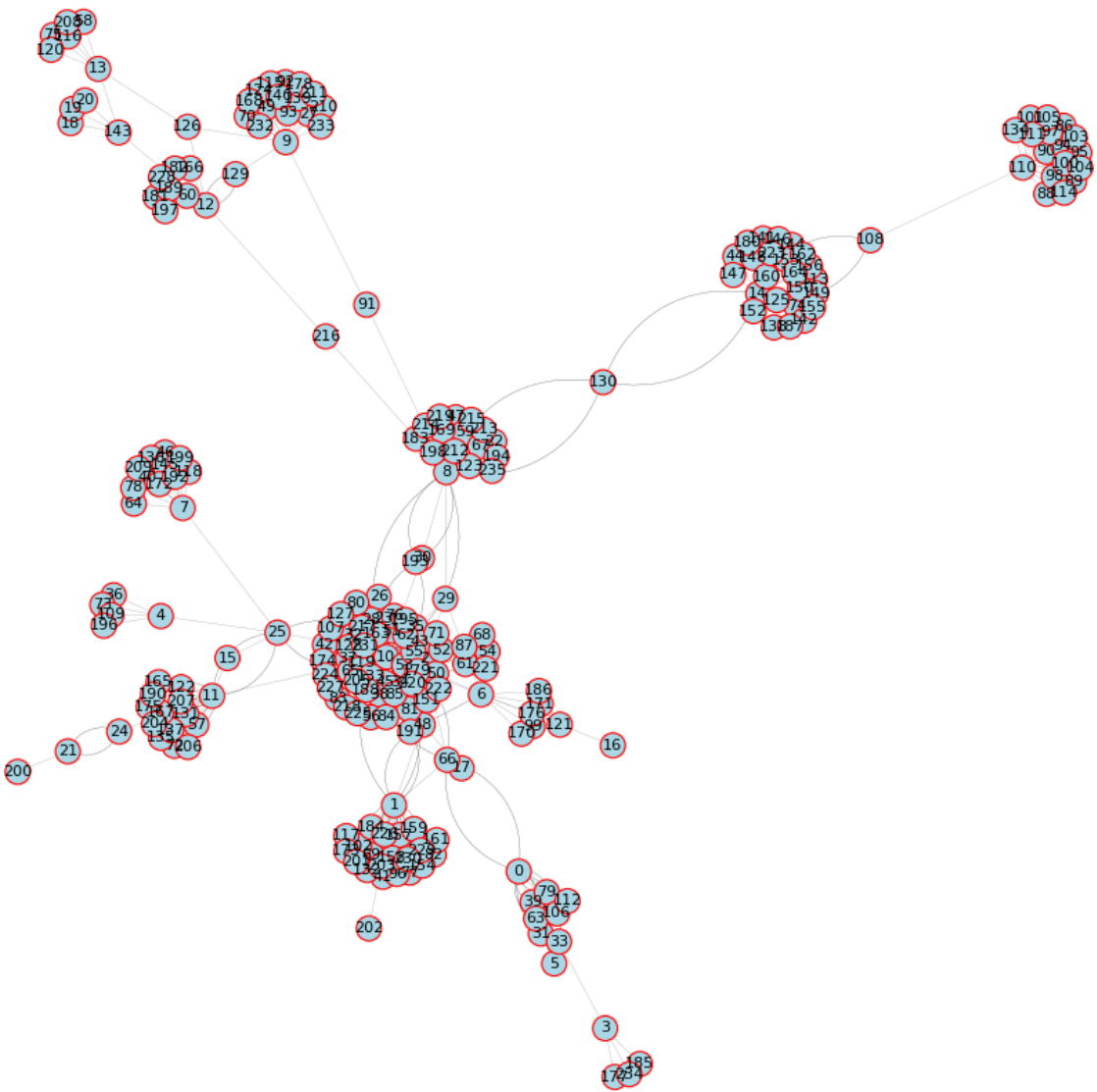
4、社群可视化

```
import igraph as ig
import matplotlib.pyplot as plt

sub_g = partition.subgraph(73)

fig, ax = plt.subplots(figsize=(12, 12))
ig.plot(sub_g,
        #bbox=(500, 500),
        target = ax,
        vertex_color="lightblue",
        vertex_shape='circle',
        vertex_frame_color='red',
        vertex_frame_width=0.8,
        vertex_label=range(sub_g.vcount()),
        vertex_label_size = 8,
        #edge_color='gray',
        edge_width = 0.1,
        layout_kamada_kawai='kk',
        #margin=10,
        vertex_size=18
        )
plt.show()
```





发布于 2024-08-04 23:28 · 浙江

关系网络 复杂网络



未登录用户

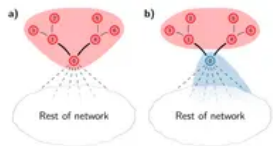
14 条评论

默认 最新



饲养员

请问这里为什么 0 这个点会移到别的地方啊？按照 louvain 不应该是把红色社区看成一个超顶点，要移也是这整个超顶点吧，为什么会单独出去一个点？🤔



08-27 · 江苏

回复 喜欢



寻仙

leidenalg 的 find_partition 方法会自动进行多层次的社区检测吗？还是需要我们手动循环调用 find_partition 来进行多层次的社区划分？

04-16 · 浙江

回复 喜欢



nenos

数据集能提供下吗

02-28 · 广东

回复 喜欢



Misaka77 · 小伍哥聊风控

你好能提供数据集吗，已三连

11-17 · 湖南

回复 喜欢



小伍哥聊风控 作者

可以的呢这边好像发不了

02-28 · 浙江

回复 喜欢



Diamond

请问Leiden算法可以对加权有向网络进行社区检测吗？

01-12 · 浙江

回复 喜欢



土拔鼠 · 把你的计算机

请问可以直接在gephi里用这个算法进行社区检测吗

05-07 · 河南

回复 喜欢



把你的计算机

可以的

03-06 · 浙江

回复 喜欢



momo

请问动态社区检测用什么算法🤔🤔🤔🤔🤔

2024-10-22 · 安徽

回复 喜欢



小伍哥聊风控 作者

这个没怎么研究

2024-10-23 · 浙江

回复 1



邹欢

想问一下不用nx的情况下，用la包可以计算pagerank这些吗

2024-10-11 · 重庆

回复 喜欢



三年二班

请问Louvain算法每个步骤不是会将节点凝聚为超级节点吗？为什么还会继续移动节点0呢？

2024-09-23 · 上海

回复 喜欢



TQTQ

同问

2024-11-03 · 广东

回复 喜欢



Giselle

不知道你们有没有需要使用🤔🤔🤔



2024-08-14 · 浙江

回复 喜欢



未登录用户

推荐阅读

社区发现算法综述

1 前言 《Network Community Detection: A Review and Visual Survey》 目前我能在 arxiv 上找到的最新的关于社区发现算法系列的综述文了。2 社区发现 现代网络在规模、多样性和复杂性上呈指...

马东什么 发表于图算法-时 ...

万物皆网络，万字长文详解社区发现算法Leiden - 知乎

LFIM	2009	$O(n^2)$
EAGLE	2009	$O(n^2s)$
GIS	2009	$O(n^2)$
HANP	2009	$O(m)$
GCE	2010	$O(mh)$
COPRA	2010	$O(m \log(\frac{vm}{n}))$
NIME	2010	$O(Kn^2)$

复杂网络社区发现算法汇总

烂泥 发表于Compl...

用户行为路径分析（附Python代码）

用户行为路径分析是什么？用户路径分析追踪用户从某个开始事件直到结束事件的行为路径，即对用户流向进行监测。用户行为路径分析有什么用？用户个体动机虽然千差万别，但是海量用户行为的...

侯永胜 发表于数据分析实 ...

用户表征（User Representation） 论文学习笔记

《One Person, One Model, OneWorld: Learning Continual User Representation without Forgetting》方法：Conure 2021 SIGIR 腾讯 西湖大学 Fajie Yuan 用持续学习做用户表征，文章提到...

云峰璇玑